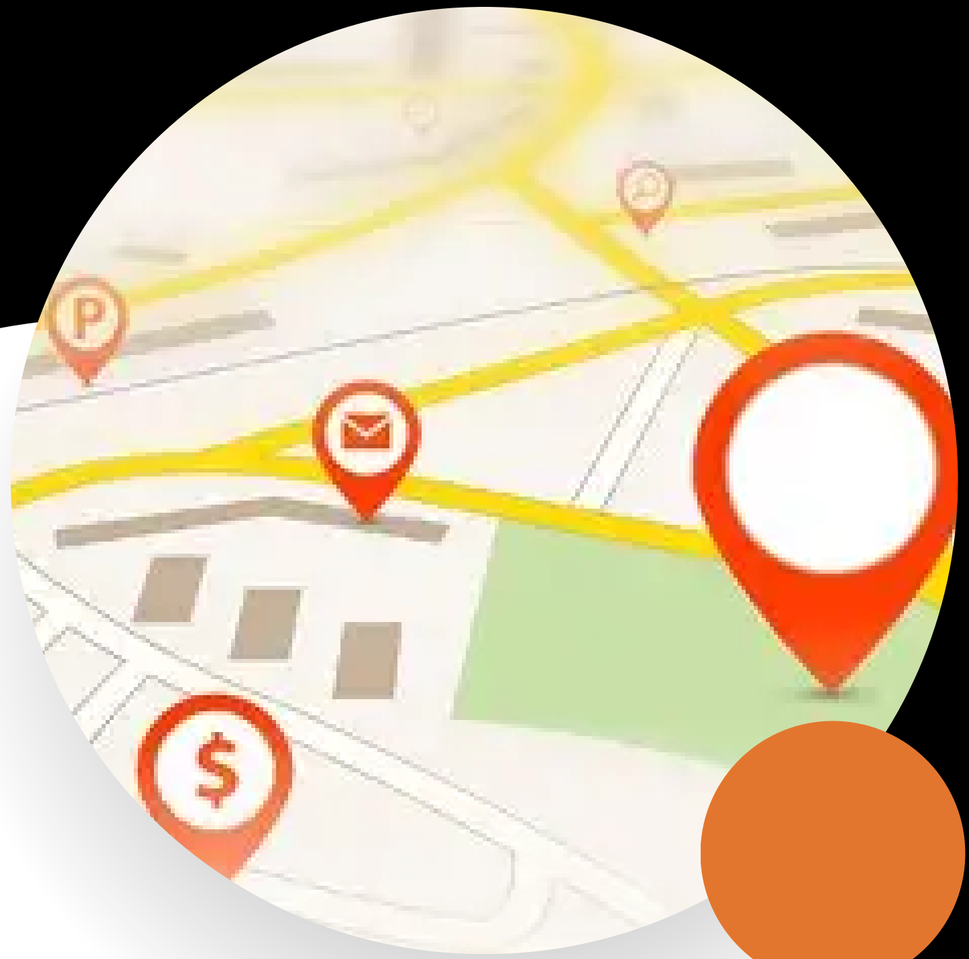


TỐI ƯU HÓA HỆ THỐNG GIAO HÀNG BẰNG THUẬT TOÁN DIJKSTRA

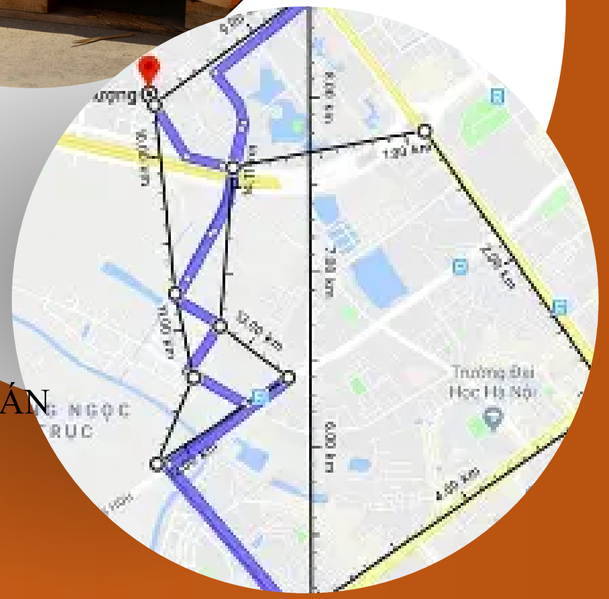
Nhóm 6

NEXT SLIDE ⟶

Giới Thiệu Chung

Đặt vấn đề

- Tối ưu hóa lộ trình giao hàng là yếu tố quan trọng trong logistics & chuỗi cung ứng
- Mục tiêu: giảm chi phí vận hành và thời gian vận chuyển
- Bài toán: tìm đường đi ngắn nhất giữa các kho trong mạng lưới giao thông phức tạp
- Yêu cầu: áp dụng thuật toán hiệu quả và chính xác



Mục tiêu

- KỸ THUẬT: CÀI ĐẶT THÀNH CÔNG THUẬT TOÁN DIJKSTRA BẰNG C++
- ỨNG DỤNG: XÂY DỰNG PHẦN MỀM MÔ PHỎNG GIAO HÀNG, TÍNH LỘ TRÌNH TỐI ƯU GIỮA CÁC KHO
- HỌC THUẬT: PHÂN TÍCH VÀ SO SÁNH HIỆU NĂNG CỦA CÁC CẤU TRÚC DỮ LIỆU KHI TRIỂN KHAI THUẬT TOÁN

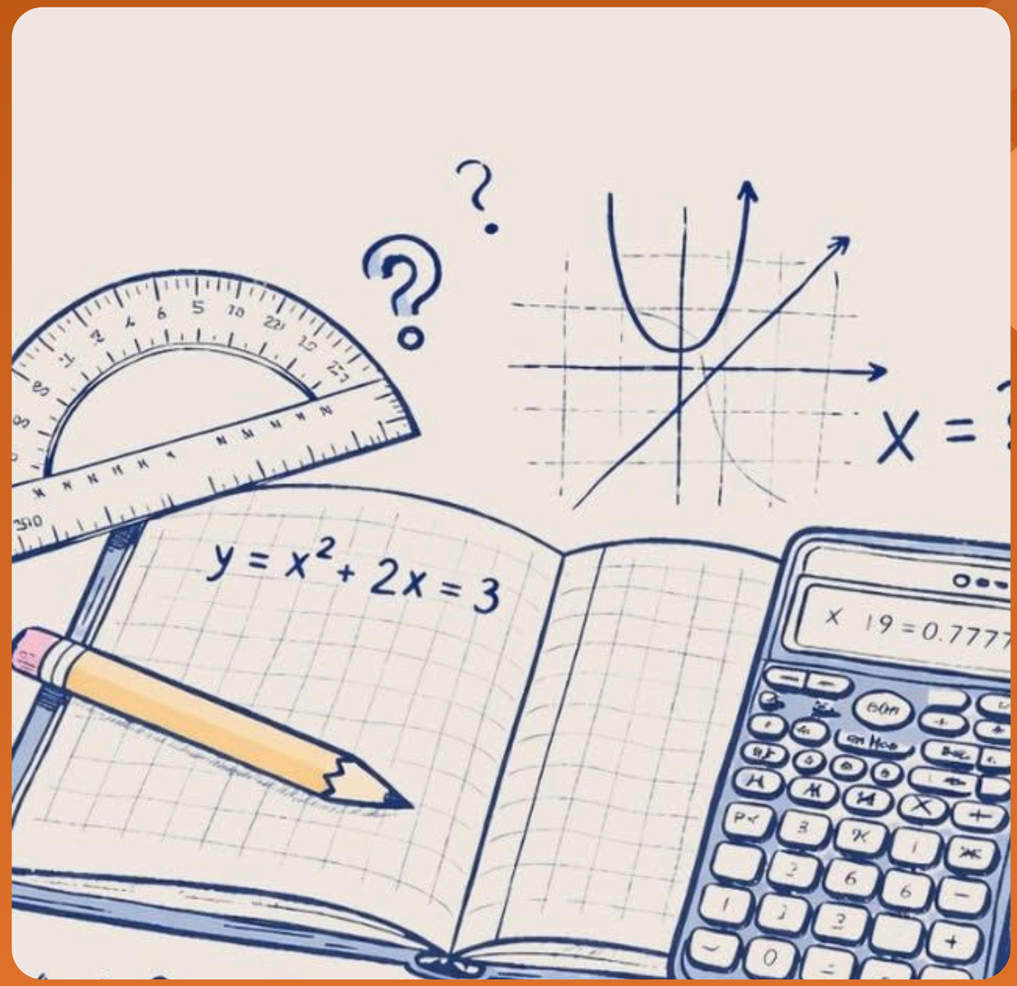
Cơ sở lý thuyết

- MÔ HÌNH HÓA ĐỒ THỊ (GRAPH MODELING)
- NGUYÊN LÝ THUẬT TOÁN DIJKSTRA
- ĐỘ PHỨC TẠP (COMPLEXITY ANALYSIS)
- NGUYÊN LÝ RELAXATION

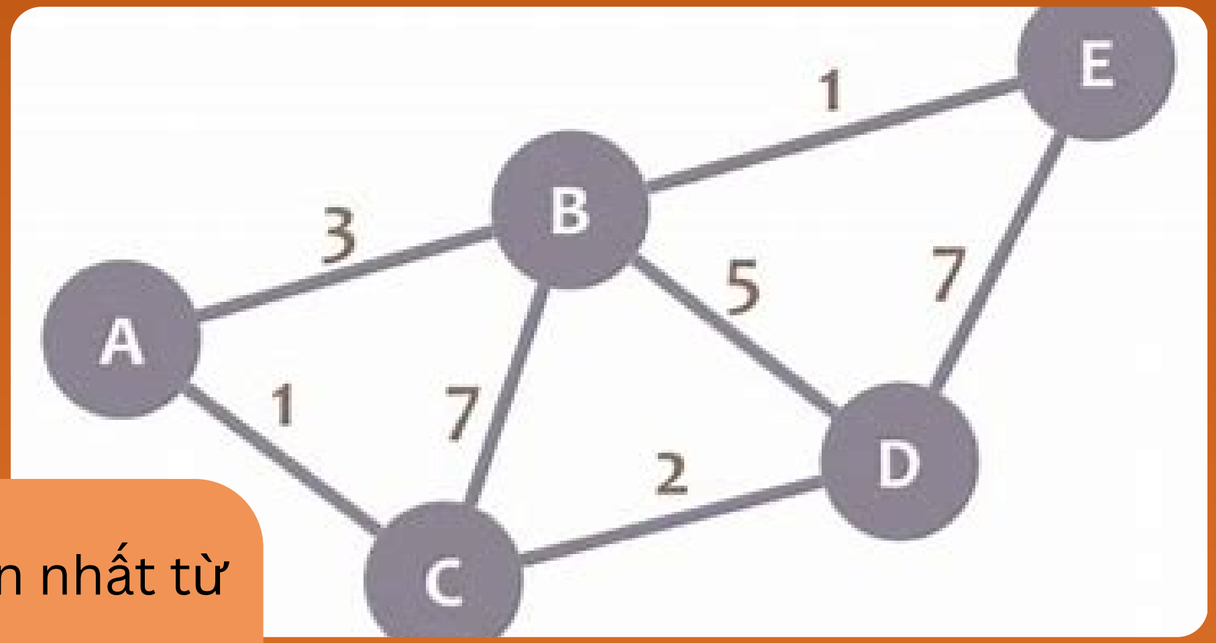
Mô hình hóa đồ thị (Graph Modeling)

Hệ thống giao hàng được đại diện bởi đồ thị có trọng số G = (V, E, W):

- V (Vertices): Tập hợp các kho bãi hoặc điểm giao hàng.
- E (Edges): Tập hợp các tuyến đường kết nối.
- W (Weights): Giá trị đại diện cho khoảng cách hoặc thời gian di chuyển (luôn dương).



Thuật toán Dijkstra



- Là thuật toán giải bài toán đường đi ngắn nhất từ một nguồn
- Áp dụng cho đồ thị có trọng số không âm
- Nếu tồn tại trọng số âm, thuật toán có thể cho kết quả sai

Độ phức tạp (Complexity Analysis)

- Adjacency list + min-heap: O((V + E) log V)
- Adjacency matrix + linear scan: O(V²)

→ Heap hiệu quả hơn khi số cạnh lớn và đồ thị thưa

Nguyên lý Relaxation

- Dùng để cập nhật khoảng cách ngắn nhất từ đỉnh nguồn đến các đỉnh khác
- Với mỗi cạnh (u, v):
  Nếu: dist[v] > dist[u] + w(u, v)
  Thì: cập nhật dist[v] = dist[u] + w(u, v)
- Lặp lại cho đến khi không còn cập nhật

→ Đảm bảo tìm được đường đi ngắn nhất tối ưu

Phân tích bài toán

MÔ HÌNH HỆ THỐNG GIAO HÀNG

DỮ LIỆU ĐẦU VÀO (INPUT)

DỮ LIỆU ĐẦU RA (OUTPUT)

RÀNG BUỘC VÀ GIẢ ĐỊNH

MÔ HÌNH HỆ THỐNG GIAO HÀNG:

Đỉnh (Vertex):

đại diện cho kho bãi hoặc điểm giao hàng

Cạnh (Edge):

đại diện cho tuyến đường kết nối giữa các điểm

Trọng số (Weight):

chi phí, khoảng cách hoặc thời gian di chuyển

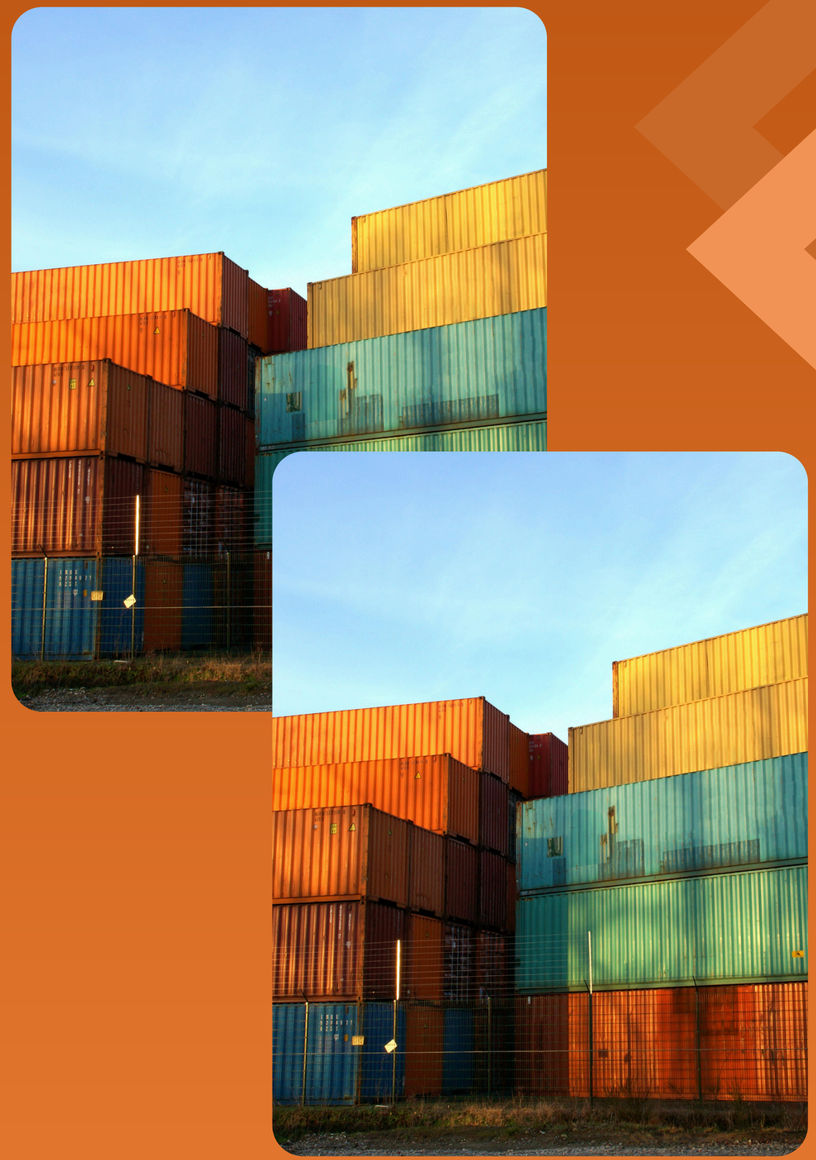
INPUT/OUTPUT

INPUT:

- Tập đỉnh và tập cạnh của đồ thị
- Trọng số dương cho mỗi cạnh
- Đỉnh nguồn (source) và đỉnh đích (target)

OUTPUT:

- Đường đi ngắn nhất từ source đến target
- Tổng chi phí của đường đi
- Thông báo nếu không tồn tại đường đi



Ràng Buộc Và Giả Định

RÀNG BUỘC VÀ GIẢ ĐỊNH:

- Không tồn tại trọng số âm
- Dữ liệu đầu vào hợp lệ (đúng chỉ số đỉnh)
- Có thể tồn tại nhiều đường đi giữa hai đỉnh → cần chọn đường tối ưu nhất

Thiết Kế Thuật Toán

Ý tưởng thuật toán &
Chiến lược tham lam

Luồng xử lý& Truy vết đường đi

Các biến thể cài đặt&
Cách thực hiện

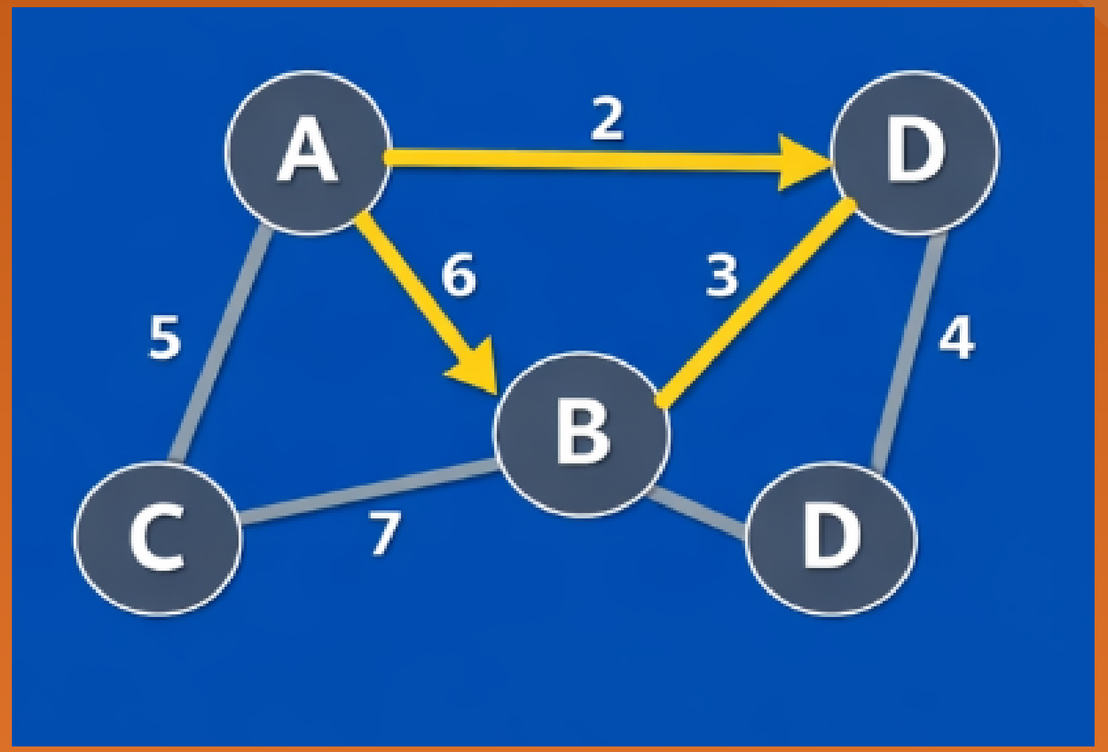
Đánh giá thuật toán

Ý tưởng thuật toán & Chiến lược tham lam

Ý tưởng thuật toán Dijkstra

- Tìm đường đi ngắn nhất từ đỉnh nguồn đến các đỉnh khác
- Tại mỗi bước:
  Chọn đỉnh có khoảng cách nhỏ nhất (chưa xét)
  Cập nhật khoảng cách đến các đỉnh kề (Relaxation)

→ Lặp lại cho đến khi xét hết các đỉnh



CHIẾN LƯỢC THAM LAM (GREEDY)

- luôn chọn đỉnh gần nhất tại thời điểm hiện tại
- Quyết định cục bộ → dẫn đến tối ưu toàn cục

Cấu trúc dữ liệu

- Graph: adjacency list
- MatrixGraph: adjacency matrix
- distance[], parent[], visited[]

Các biến thể cài đặt

Adjacency List + Min-Heap (O(E log V))

- Sử dụng priority queue để chọn đỉnh nhỏ nhất
- Cập nhật và đưa vào heap

Adjacency Matrix (O(V²))

- Duyệt tuyến tính để chọn đỉnh nhỏ nhất
- Cập nhật trực tiếp trên ma trận

Cách thực hiện

Adjacency List + Min-Heap (O(E log V))

- Khởi tạo distance[source] = 0
- Đưa source vào min-heap
- Lặp:
- Lấy đỉnh u có distance nhỏ nhất từ heap
- Với mỗi đỉnh kề v:
- Nếu dist[u] + w(u,v) < dist[v]
  → cập nhật dist[v]
  → đưa v vào heap
- Kết thúc khi heap rỗng hoặc đến đích

ADJACENCY MATRIX (O(V²))

- Khởi tạo distance[source] = 0
- visited = false
- Lặp:
- Chọn đỉnh u chưa xét có distance nhỏ nhất (duyệt tuyến tính)
- Đánh dấu u đã xét
- Với mọi đỉnh v:
- Nếu có cạnh và chưa xét:
- Nếu dist[u] + w(u,v) < dist[v]
- → cập nhật dist[v]
- Kết thúc khi xét hết các đỉnh

Luồng xử lí& Truy vết đường đi

LUỒNG XỬ LÝ:

- Chạy Dijkstra (heap)
- Chạy Dijkstra (matrix)
- Cross-check kết quả

TRUY VẾT ĐƯỜNG ĐI:

- Dùng parent[] từ target → source

Đánh Giá Thuật Toán

Độ phức tạp:

- O(V^2) (ma trận kề)
- O(E log V) (dùng hàng đợi ưu tiên)

Ưu điểm:

- Đơn giản, dễ cài đặt
- Hiệu quả với đồ thị trọng số không âm

Hạn chế:

- Không áp dụng cho đồ thị có cạnh âm

Cài đặt chương trình

Kiến trúc phần mềm

Chức năng chính

Cài đặt thuật toán Dijsktra

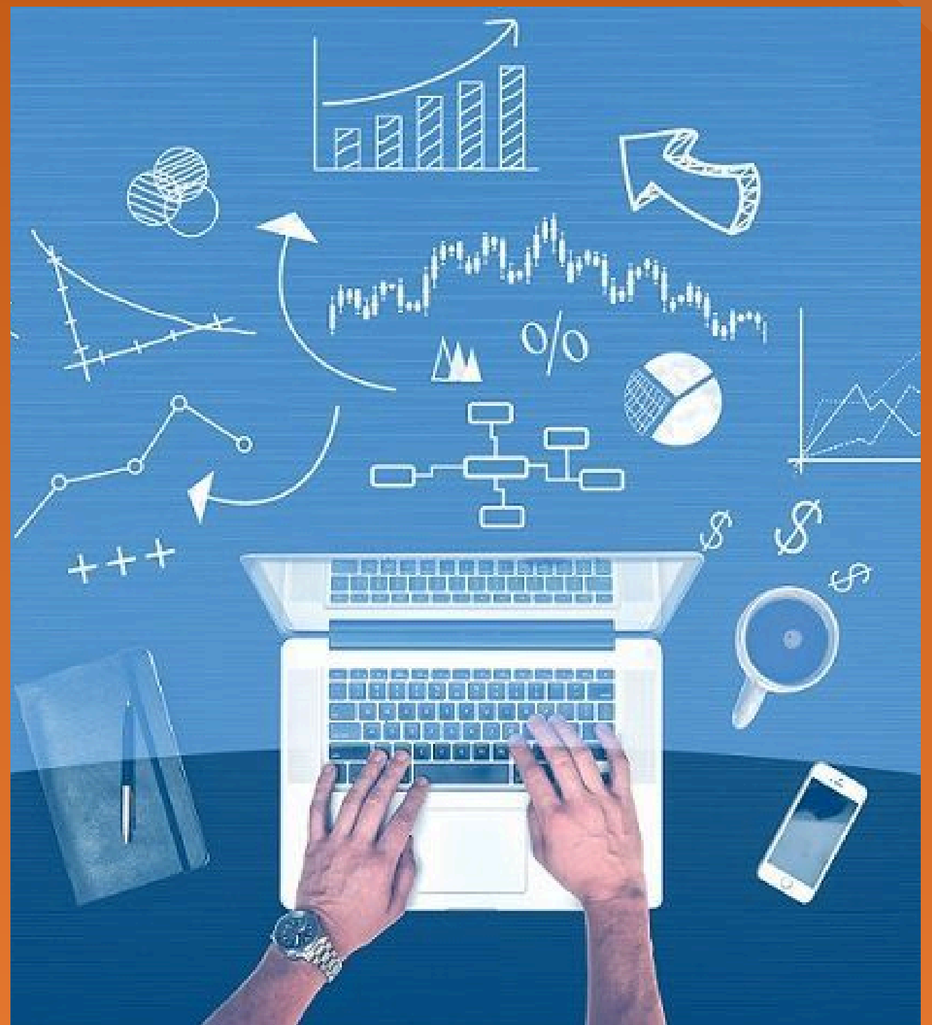
KIẾN TRÚC PHẦN MỀM

Công nghệ:

- C++20/23
- WinUI 3 (C++/WinRT)

Kiến trúc:

- Graph.* → lưu đồ thị
- MatrixGraph.* → ma trận
- DijkstraSolver.* → thuật toán + benchmark
- MainWindow → giao diện



Chức năng chính

01 Nhập dữ liệu

02 Chọn source/target

03 Chạy thuật toán

04 Hiển thị đường đi + chi phí

Cài đặt thuật toán Dijsktra

Thử Nghiệm Và Đánh Giá

Test Case

Mục đích thử nghiệm

Kiểm tra tính đúng đắn của thuật toán trong nhiều trường hợp khác nhau

Đánh giá hiệu năng giữa các cách cài đặt (heap vs matrix)

Xác minh sự phù hợp giữa kết quả thực tế và lý thuyết

Testcase:

TC1: Có đường đi

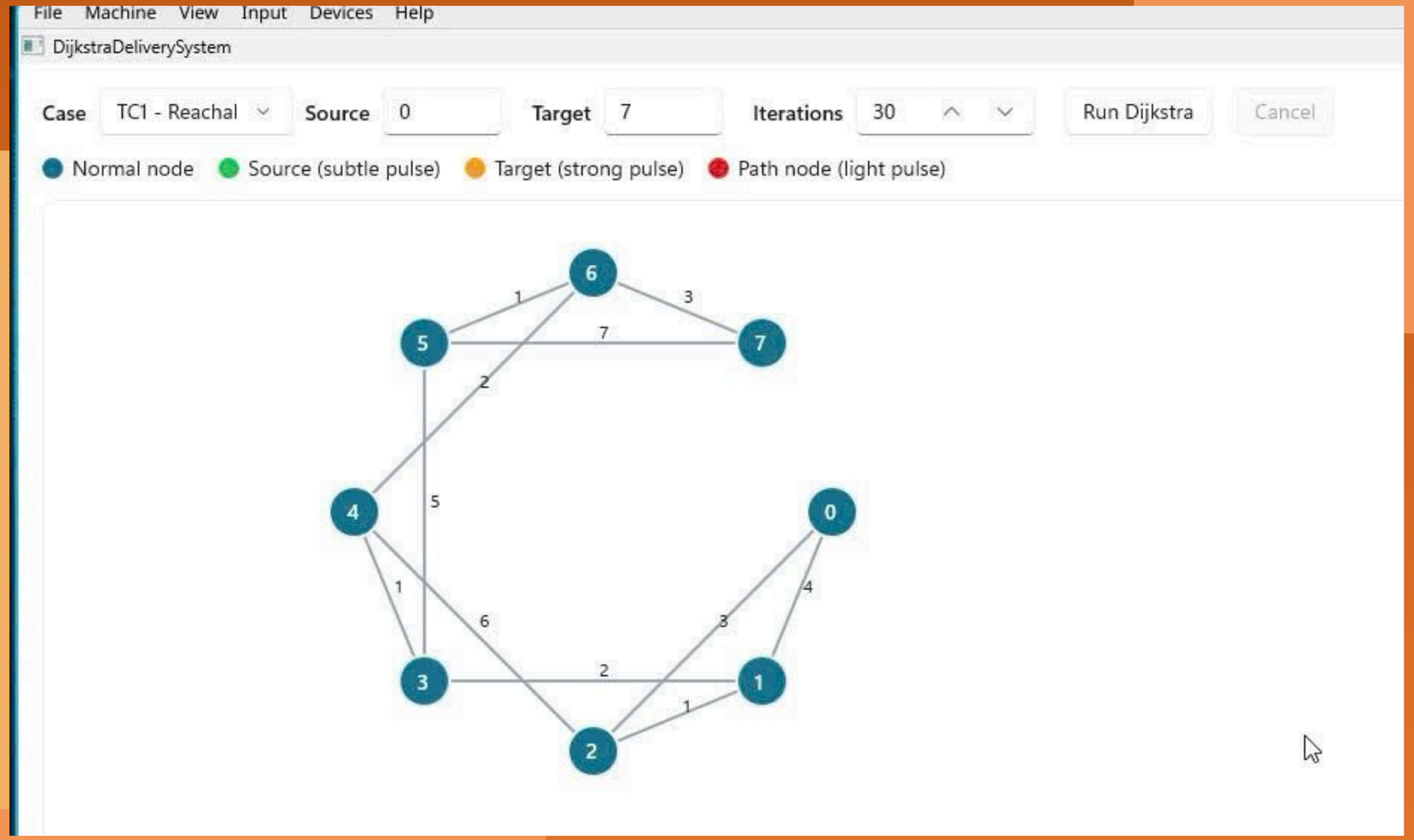
TC2: Nhiều đường đi

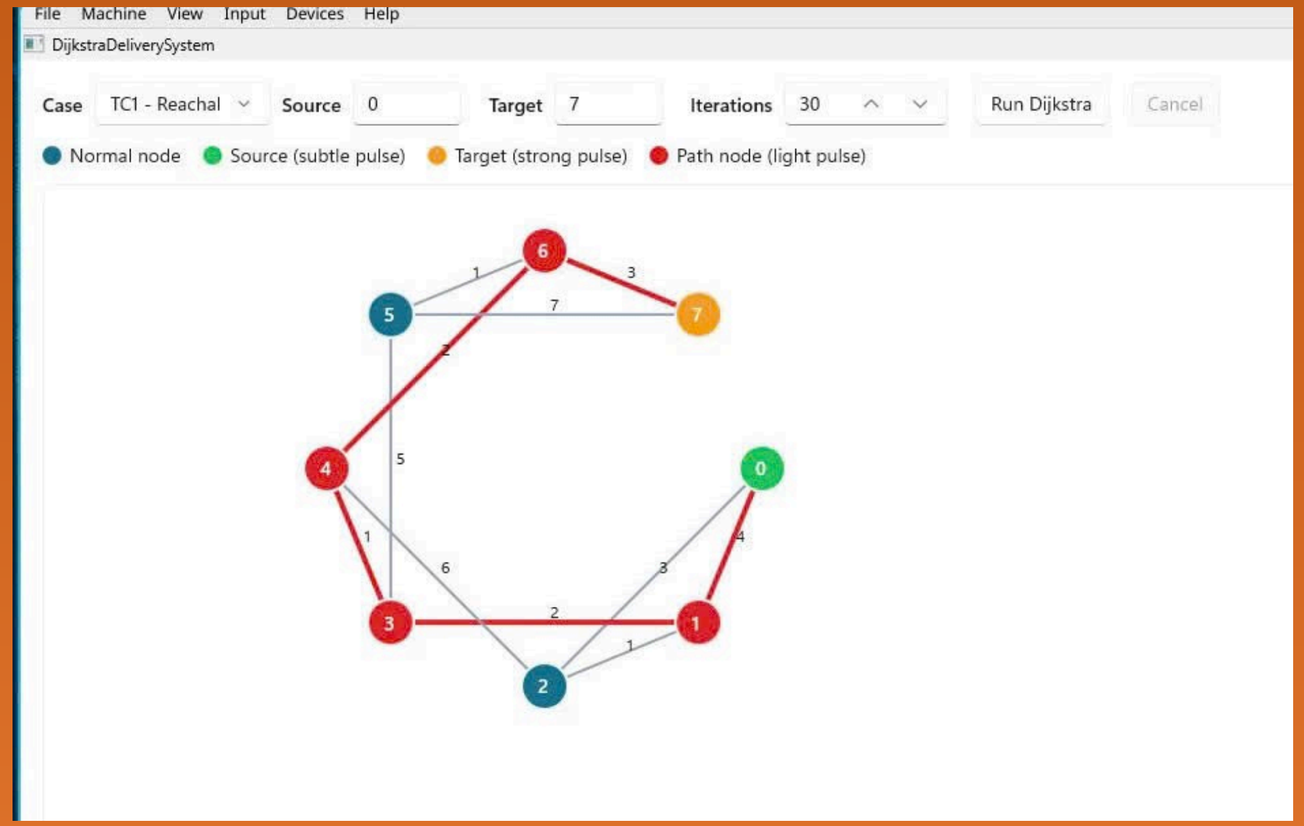
TC3: Không có đường đi

TC4: source = target

TC5: đồ thị lớn

Giao diện chính

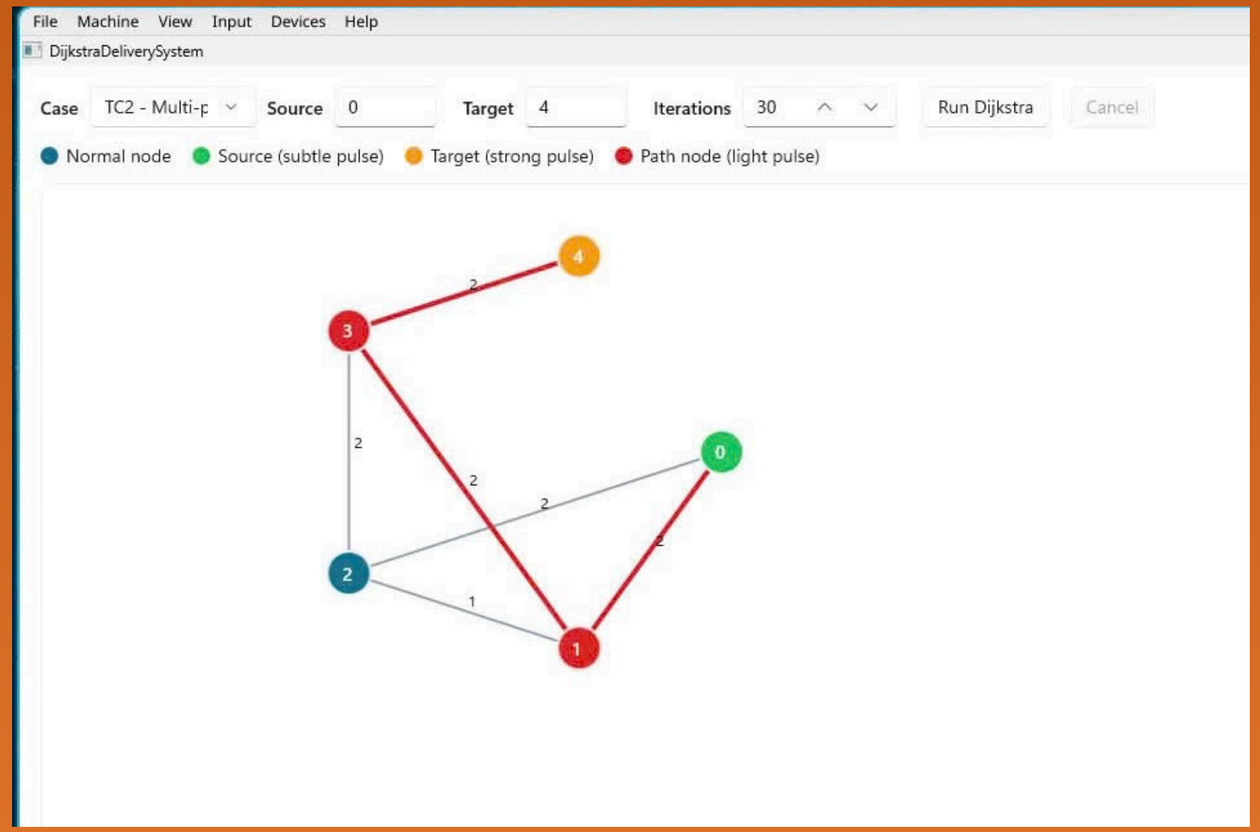




Test case: TC1
Điểm bắt đầu: 0, Điểm đích: 7
Khoảng cách ngắn nhất: 12
Đường đi: 0 → 1 → 3 → 4 → 6 → 7
Khoảng cách kỳ vọng: 12 (ĐẠT)

Benchmark:
Danh sách kề + heap (30 lần chạy):
Tốt nhất: 0µs, Trung bình: 0.10µs, Tệ nhất: 2µs
Ma trận kề + duyệt tuyến tính (30 lần chạy):
Tốt nhất: 0µs, Trung bình: 0.07µs, Tệ nhất: 1µs

Đối chiếu kết quả (cross-check):
So sánh khoảng cách: ĐẠT



Test case: TC2

Điểm bắt đầu: 0, Điểm đích: 4

Khoảng cách ngắn nhất: 6

Đường đi: 0 → 1 → 3 → 4

Khoảng cách kỳ vọng: 6 (ĐẠT)

Benchmark:

Danh sách kề + heap (30 lần chạy):

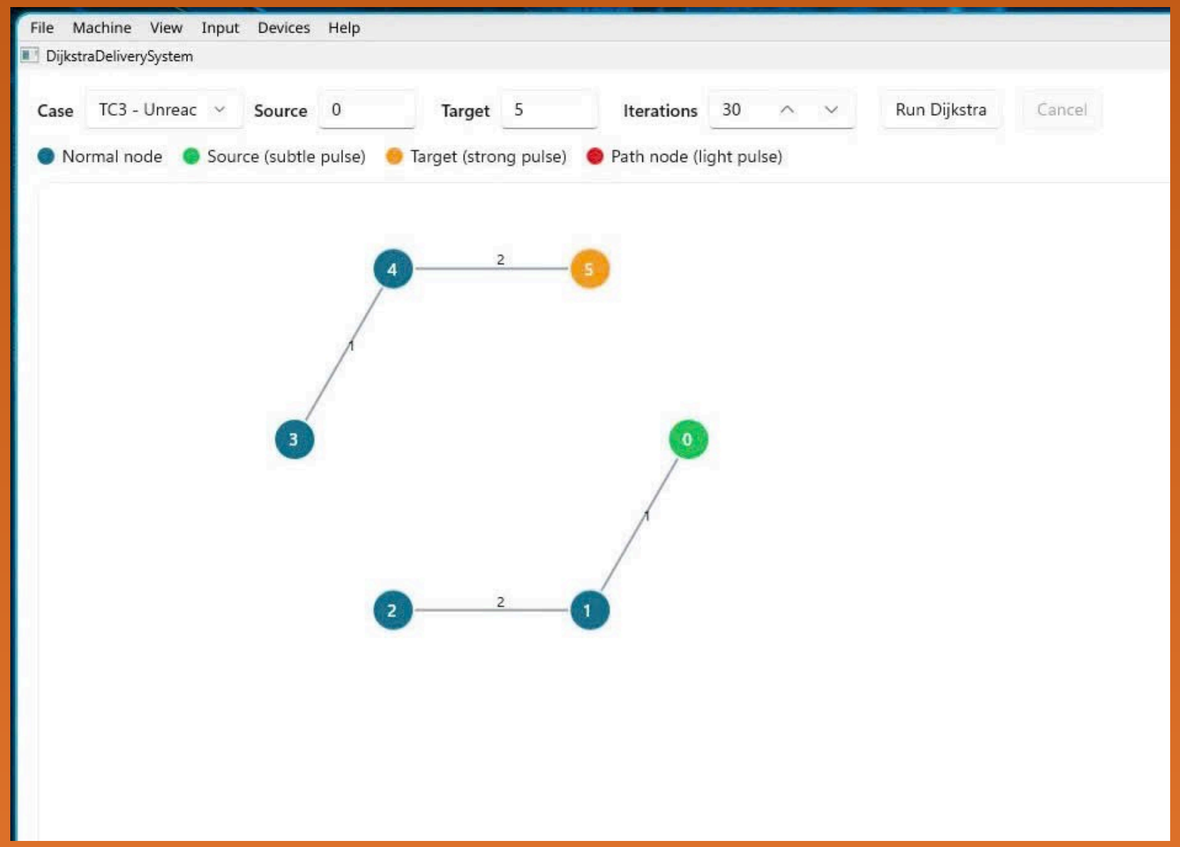
Tốt nhất: 0µs, Trung bình: 0.07µs, Tệ nhất: 1µs

Ma trận kề + duyệt tuyến tính (30 lần chạy):

Tốt nhất: 0µs, Trung bình: 0.07µs, Tệ nhất: 1µs

Đối chiếu kết quả (cross-check):

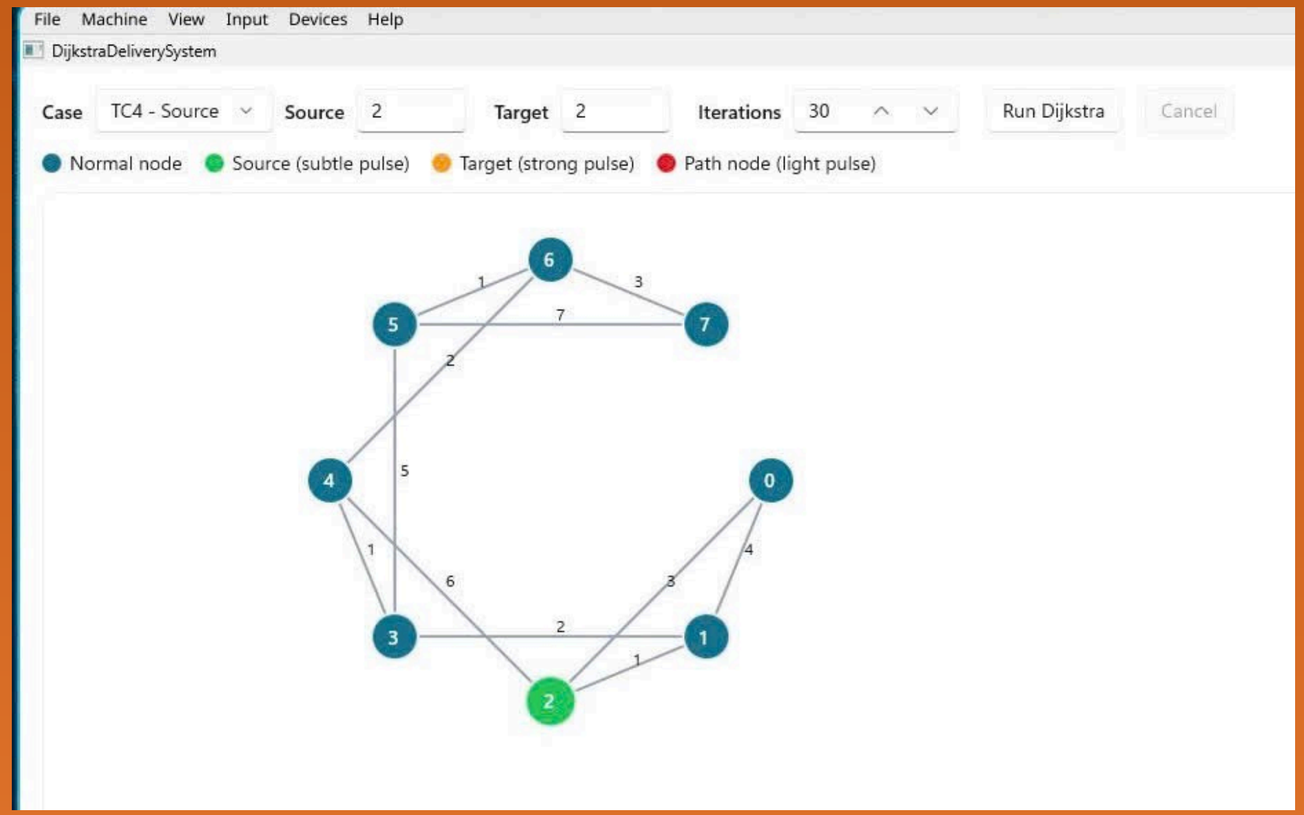
So sánh khoảng cách: ĐẠT



Test case: TC3
Điểm bắt đầu: 0, Điểm đích: 5
Không tìm thấy đường đi

Benchmark:
Danh sách kề + heap (30 lần chạy):
Tốt nhất: 0µs, Trung bình: 0.07µs, Tệ nhất: 1µs
Ma trận kề + duyệt tuyến tính (30 lần chạy):
Tốt nhất: 0µs, Trung bình: 0.07µs, Tệ nhất: 1µs
Đối chiếu kết quả (cross-check):

Đối chiếu kết quả (cross-check):
So sánh khoảng cách: ĐẠT



Test case: TC4

Điểm bắt đầu: 2, Điểm đích: 2

Khoảng cách ngắn nhất: 0

Đường đi: 2

Khoảng cách kỳ vọng: 0 (ĐẠT)

Benchmark:

Danh sách kề + heap (30 lần chạy):

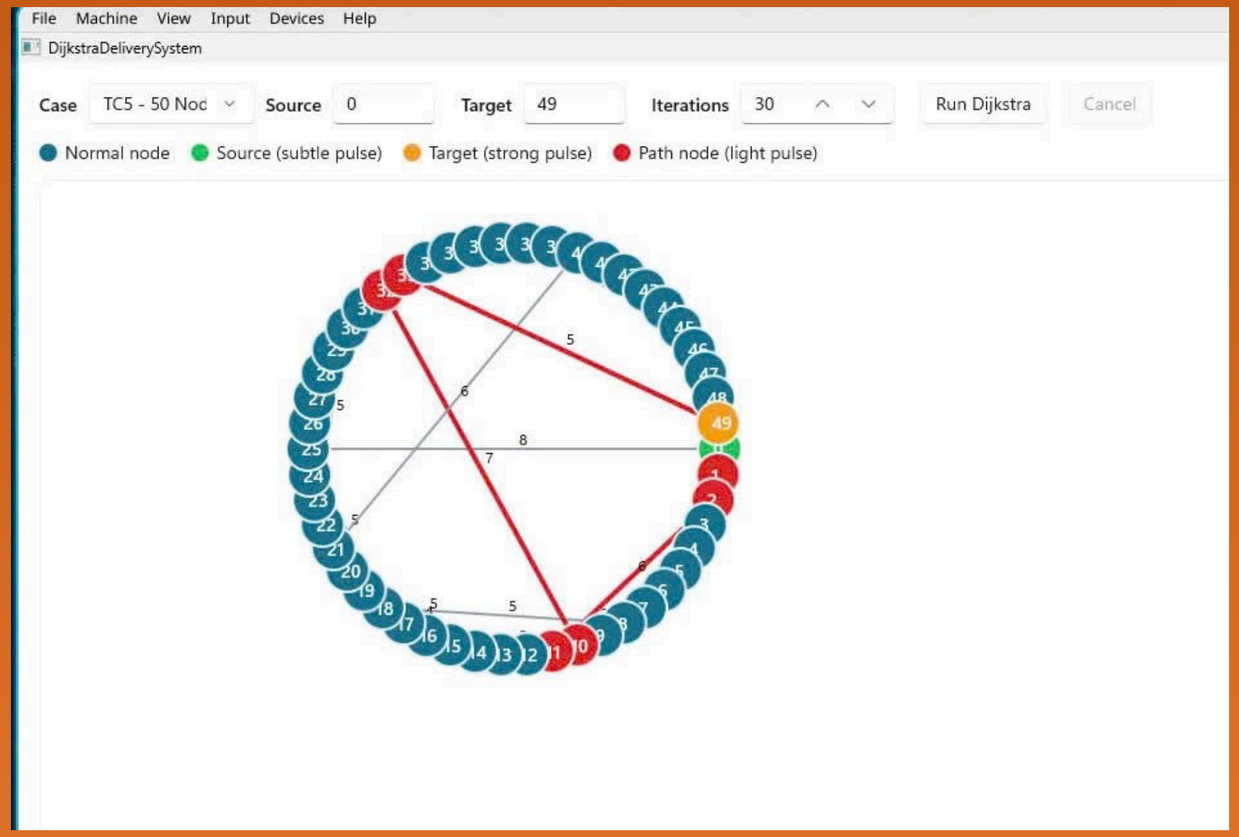
Tốt nhất: 0µs, Trung bình: 0.00µs, Tệ nhất: 0µs

Ma trận kề + duyệt tuyến tính (30 lần chạy):

Tốt nhất: 0µs, Trung bình: 0.00µs, Tệ nhất: 0µs

Đối chiếu kết quả (cross-check):

So sánh khoảng cách: ĐẠT



Test case: TC5

Điểm bắt đầu: 0, Điểm đích: 49

Khoảng cách ngắn nhất: 25

Đường đi: 0 → 1 → 2 → 11 → 10 → 32 → 33 → 49

Benchmark:

Danh sách kề + heap (30 lần chạy):

Tốt nhất: 2µs, Trung bình: 2.80µs, Tệ nhất: 10µs

Ma trận kề + duyệt tuyến tính (30 lần chạy):

Tốt nhất: 6µs, Trung bình: 9.17µs, Tệ nhất: 23µs

Đối chiếu kết quả (cross-check):

So sánh khoảng cách: ĐẠT

Kết Luận

KẾT QUẢ ĐẠT ĐƯỢC:

- Giải đúng bài toán đường đi ngắn nhất.
- Có so sánh đa chiều theo cấu trúc dữ liệu.

HẠN CHẾ HIỆN TẠI:

- Chưa mô hình hóa traffic real-time và ràng buộc

Hướng mở rộng

Bổ sung:
Bellman-Ford (cạnh âm)
A* (tối ưu hơn)

Cải tiến:

Nhập dữ liệu từ file JSON
Tích hợp bản đồ thực tế

Mở rộng:

Mô phỏng hệ thống logistics
Xử lý dữ liệu lớn

Thank You!

self.file = None
self.fingerprints = set()
self.logdupes = True
self.debug = debug
self.logger = logging.
if path:
self.file = open(os.path.
self.file.seek(0)
self.fingerprints.
@classmethod
def from_settings(cls, settings):
debug = settings.getbool("
return cls(job_dir(settings),
def request_seen(self, request):
fp = self.request_fingerprint(
if fp in self.fingerprints:
return True